

Acknowledgement

I would like to express my sincere gratitude to the Department of Computer Engineering, Harimohan Khatri Sir for providing the opportunity to work on this Individual Course Project under the Image Processing and Pattern Recognition (IPPR) course.

I also extend my appreciation to my peers and family for their encouragement and support during the completion of this work .

Bishesh KC (023-318)

Abstract

This project presents a Bike Helmet Detection System developed as part of the Image Processing and Pattern Recognition (IPPR) course. The system detects whether a person is wearing a helmet from uploaded images using a deep learning approach. A Convolutional Neural Network (CNN) was trained from scratch on a custom dataset of 1,522 images collected from Google Images, Kaggle, and Roboflow. The dataset was organized into two classes: with_helmet and without_helmet. The CNN architecture consists of five convolutional layers with batch normalization, dropout, and fully connected layers for classification. The model was trained using PyTorch on an NVIDIA RTX 4050 GPU and achieved a test accuracy of 62.86%, with 98% recall for the with_helmet class. The system was deployed as a Flask web application, allowing users to upload images and receive predictions with confidence scores. This project demonstrates that deep learning models trained from scratch can effectively solve real-world image classification problems. Future work includes dataset balancing, expanding the dataset, and adding live video processing support.

(Keywords: Bike Helmet Detection, Convolutional Neural Network, Deep Learning, Image Classification, PyTorch, GPU Acceleration, Flask Web Application)

Contents

Acknowledgement.....	i
Abstract	ii
Table of tables	v
Chapter 1 :Introduction	1
1.1 Problem definition.....	1
1.2 Objectives.....	1
1.3 Scope	2
1.3.1 Features Included	2
1.3.2 Dataset Used.....	2
1.3.3 Limitations	3
Chapter 2 Literature Review	3
2.1 CNN Architechure Components	3
2.1.1 Batch Normalization	3
2.1.2 Dropout Regularization	4
2.1.3 Relu function	4
2.1.4 Adam Optimizer	4
2.2 Pytorch	4
2.3 OpenCV.....	4
2.4 GPU Acceleration and CUDA	4
2.5 Flask	4
Chapter 3: System Analysis & Design	5
3.1 Introduction	5
3.1.1 Requirement Analysis	5
3.2 Dataset Descriptions.....	6
3.2.1 Dataset Sources	6
3.2.2 Dataset Structure	6
3.2.3 Data preprocessing	7
3.3 Flowchart(making of the model).....	8
CHAPTER 4: IMPLEMENTATION	10
4.1 Tools Used.....	10
4.2 Modules Used.....	10

4.3 Algorithms Used.....	11
Chapter 5 Testing	13
5.1 Test cases	13
5.2 Accuracy Tables	13
5.2.1 Dataset Split	13
5.2.2 Overall Performance Metrics	14
5.2.3 Per-Class Performance	14
5.2.4 Confusion Matrix	15
5.2.5 Comparison of Classifiers	15
5.3 Results	15
5.3.1 Key Observations	15
CHAPTER 6: CONCLUSION.....	17
6.1 Introduction	17
6.2 Achievements	17
6.3 Limitations	18
6.4 Future Scope.....	19
References	20
Appendix	22

Table of tables

Table 1:dataset All images were resized to 224×224 pixels, normalized, and augmented during training using random rotation, horizontal flipping, color jitter, and affine transformations to simulate real-world variation.	2
Table 2: Dataset.....	7
Table 3:Technology used.....	10
Table 4:test cases	13
Table 5:performance metrics	14
Table 6:Per class.....	14
Table 7:Confusion Matrix	15
Table 8:comparison	15
Table 9:Observations.....	15

Chapter 1 :Introduction

Road safety has become a major concern around the world, especially with the increasing number of motorcycle and bicycle accidents. Helmets play an important role in protecting riders from serious head injuries and reducing the risk of death during accidents. Although helmet use is legally required in many countries, people do not always follow these rules. This is particularly common in developing countries, where traffic monitoring and enforcement can be limited.

Traditionally, helmet use is monitored manually by traffic police or through human observation. However, this approach can be time-consuming, difficult to maintain continuously, and prone to human error. Monitoring a large number of roads and vehicles manually is also not practical. With recent developments in computer vision and deep learning, automated systems can provide a more efficient way of identifying helmet violations.

This project focuses on developing a Bike Helmet Detection System as part of the Image Processing and Pattern Recognition (IPPR) course. The system uses a Convolutional Neural Network (CNN) to classify images into two categories: people wearing helmets and people not wearing helmets. The CNN is trained from scratch rather than using a pre-trained model, allowing the project to demonstrate the fundamental concepts of image classification and deep learning covered in the course.

The developed system accepts static images through a web interface and analyzes them to determine whether a helmet is present. It then displays the predicted class along with a confidence score, allowing the user to understand how confident the model is in its prediction.

The complete system is implemented as an interactive web application using Flask. This makes the model easier to use even for users without technical knowledge. Overall, the project demonstrates how image processing and deep learning can be applied to a practical road-safety problem and provides a foundation for developing more advanced automated traffic monitoring systems in the future.

1.1 Problem definition

Manual monitoring of helmet use by motorcycle and bicycle riders is time-consuming, inefficient, and difficult to perform on a large scale. Therefore, there is a need for an automated system that can detect helmet violations from images. This project aims to develop a CNN-based Bike Helmet Detection System that classifies riders into helmet-wearing and non-helmet-wearing categories and provides the prediction with a confidence score through a Flask web application.

1.2 Objectives

- To develop a CNN-based system for detecting helmet usage from images.

- To classify images into helmet and no-helmet categories.
- To provide prediction results along with a confidence score.
- To integrate the trained model into a Flask web application for easy image upload and prediction.
- To demonstrate the practical application of image processing and deep learning in road safety

1.3 Scope

The system is capable of recognizing two distinct classes: with_helmet and without_helmet from images of bicycle and motorcycle riders. It supports static image input (file upload) through a web-based interface for real-time classification.

1.3.1 Features Included

- Binary image classification (helmet vs. no helmet)
- Web-based user interface for image upload and result display
- Confidence score visualization with percentage and progress bar
- REST API endpoints for programmatic access
- GPU-accelerated inference using NVIDIA CUDA
- Automatic model saving and early stopping during training
- Error handling for invalid file types and processing failures

1.3.2 Dataset Used

A custom dataset of 1,522 images was curated from multiple sources including Google Images, Kaggle, and Roboflow public datasets, since pre-trained models and externally sourced datasets are not permitted. The dataset is organized into two classes:

Split	with_helmet	without_helmet	Total
Training	937	327	1,264
Validation	127	26	153
Test	64	41	105
Total	1,128	394	1,522

Table 1:dataset

Figure 1:flowchart of making of the project	9
Figure 2:Accuracy,loss and gap	16

All images were resized to 224×224 pixels, normalized, and augmented during training using random rotation, horizontal flipping, color jitter, and affine transformations to simulate real-world variation.

1.3.3 Limitations

The system is designed for clear, well-lit images of bicycle and motorcycle riders with reasonable contrast and visibility. Performance may degrade with:

- Heavily cluttered backgrounds or partially occluded riders
- Extreme lighting conditions (very dark or overexposed images)
- Low-quality or blurry images
- Images with multiple riders where the helmet status differs
- Unusual camera angles or perspectives not represented in the training data
- Images of people wearing helmets that are visually dissimilar to those in the training dataset (e.g., construction hard hats, unusual helmet styles)

Chapter 2 Literature Review

A Convolutional Neural Network (CNN) is a deep learning architecture that automatically and adaptively learns spatial hierarchies of features from input images. The convolutional layer is the core building block of a CNN, consisting of a set of learnable filters that are convolved with the input to produce feature maps.[1] CNNs are biologically-inspired variants of multilayer perceptrons designed to mimic the behavior of a visual cortex. They are designed to recognize visual patterns directly from pixel images with minimal preprocessing.[2]

2.1 CNN Architecture Components

A typical CNN architecture consists of three types of layers: convolutional layers, pooling layers, and fully-connected layers. The first two layers perform feature extraction, while the fully-connected layer maps the extracted features into final output.[3] The convolutional layer applies a set of learnable filters to the input image. Each filter is convolved across the width and height of the input volume, computing dot products between the filter and the input and producing a 2-dimensional activation map of that filter.[4]

2.1.1 Batch Normalization

Batch Normalization is a technique that normalizes the inputs to each layer in a neural network, reducing internal covariate shift and allowing for higher learning rates, less careful initialization, and serving as a regularizer [5]. Batch Normalization enables the use of much higher learning rates and reduces the dependence on initialization. It also acts as a regularizer, eliminating the need for Dropout in some cases.[6]

2.1.2 Dropout Regularization

Dropout is a regularization technique for reducing overfitting in neural networks by preventing complex co-adaptations on training data. The term 'dropout' refers to randomly dropping out units (along with their connections) from the neural network during training. Dropout is a regularization technique for reducing overfitting in neural networks by preventing complex co-adaptations on training data. The term 'dropout' refers to randomly dropping out units (along with their connections) from the neural network during training.[7]

2.1.3 Relu function

The Rectified Linear Unit (ReLU) activation function has become the default choice for hidden layers in CNNs due to its ability to accelerate training and reduce the vanishing gradient problem.[8]

2.1.4 Adam Optimizer

Adam (Adaptive Moment Estimation) is an optimization algorithm that can be used to update network weights based on training data. It combines the advantages of two other extensions of stochastic gradient descent: AdaGrad and RMSProp.[9]

2.2 Pytorch

PyTorch is a machine learning library for Python that provides two high-level features: tensor computation (like NumPy) with strong GPU acceleration, and deep neural networks built on a tape-based automatic differentiation system.[10]

PyTorch is a Python-based scientific computing package that combines the flexibility of NumPy with the power of GPUs. It provides a fast and flexible deep learning platform that allows easy prototyping and deployment.[11]

2.3 OpenCV

Image preprocessing using OpenCV is essential for preparing images for deep learning models. Operations such as resizing, normalization, and color space conversion improve model performance and consistency.[12]

2.4 GPU Acceleration and CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA for general computing on GPUs. It enables dramatic increases in computing performance by harnessing the power of GPUs.[13]

2.5 Flask

Flask is a lightweight WSGI web application framework for Python. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.[14]

Chapter 3: System Analysis & Design

3.1 Introduction

This chapter presents the analysis and design of the Bike Helmet Detection System. The chapter covers requirement analysis, dataset description, system flowchart, and architecture diagram of the proposed system.

3.1.1 Requirement Analysis

The requirement analysis phase identifies the functional and non-functional requirements of the system.

3.1.1.1 Functional Requirements

- The system shall accept image uploads in JPG, JPEG, PNG, BMP, and WEBP formats.
- The system shall preprocess uploaded images by resizing to 224×224 pixels and normalizing pixel values.
- The system shall classify images into two categories: with_helmet and without_helmet.
- The system shall display the prediction result along with a confidence score.
- The system shall provide a web interface for image upload and result display.
- The system shall save the best-performing model weights during training.
- The system shall train a CNN model from scratch using PyTorch.
- The system shall apply data augmentation during training.
- The system shall leverage GPU acceleration for training and inference.

3.1.1.2 Non-Functional Requirements

- The system shall process a single image prediction in under 2 seconds.
- The system shall achieve a minimum validation accuracy of 80%.
- The system shall support GPU acceleration using CUDA.
- The system shall be accessible via a web browser.
- The system shall provide clear error messages for invalid inputs.
- The system shall run on Windows 10/11.
- The system shall be modular and well-documented.

3.2 Dataset Descriptions

The dataset used for training the Bike Helmet Detection System is a custom-curated collection of images of bicycle and motorcycle riders, categorized into two classes: `with_helmet` and `without_helmet`.

3.2.1 Dataset Sources

The images were collected from multiple sources:

- **Google Images** – Downloaded using batch download tools
- **Kaggle** – Public helmet detection datasets
- **Roboflow** – Curated helmet detection images
- **Manual Collection** – Photos taken from public spaces

3.2.2 Dataset Structure

The dataset follows the PyTorch ImageFolder structure:

text

Bike Helmet Detection/

```
├── train/
│   ├── with_helmet/   (937 images)
│   └── without_helmet/ (327 images)
├── test/
│   ├── with_helmet/   (64 images)
│   └── without_helmet/ (41 images)
└── valid/
    ├── with_helmet/   (127 images)
    └── without_helmet/ (26 images)
```

Dataset Statistics

Split	with_helmet	without_helmet	Total
Training	937	327	1,264

Split	with_helmet	without_helmet	Total
Validation	127	26	153
Test	64	41	105
Total	1,128	394	1,522

Table 2: Dataset

3.2.3 Data preprocessing

The following preprocessing steps are applied to all images before they are fed into the model:

- All images are resized to 224 by 224 pixels to match the input size expected by the CNN.
- Pixel values are normalized using a mean of 0.485 for the red channel, 0.456 for the green channel, and 0.406 for the blue channel, with standard deviations of 0.229, 0.224, and 0.225 respectively, which are standard normalization values used for pretrained models.
- During training, data augmentation techniques are applied to improve model generalization and reduce overfitting. These include random rotation up to 30 degrees, random horizontal flipping, color jitter to adjust brightness, contrast, and saturation, and random affine transformations.
- During validation and testing, only resizing and normalization are applied, without any random augmentation.
- A batch size of 64 images is used for efficient GPU utilization during training.

3.3 Flowchart(making of the model)

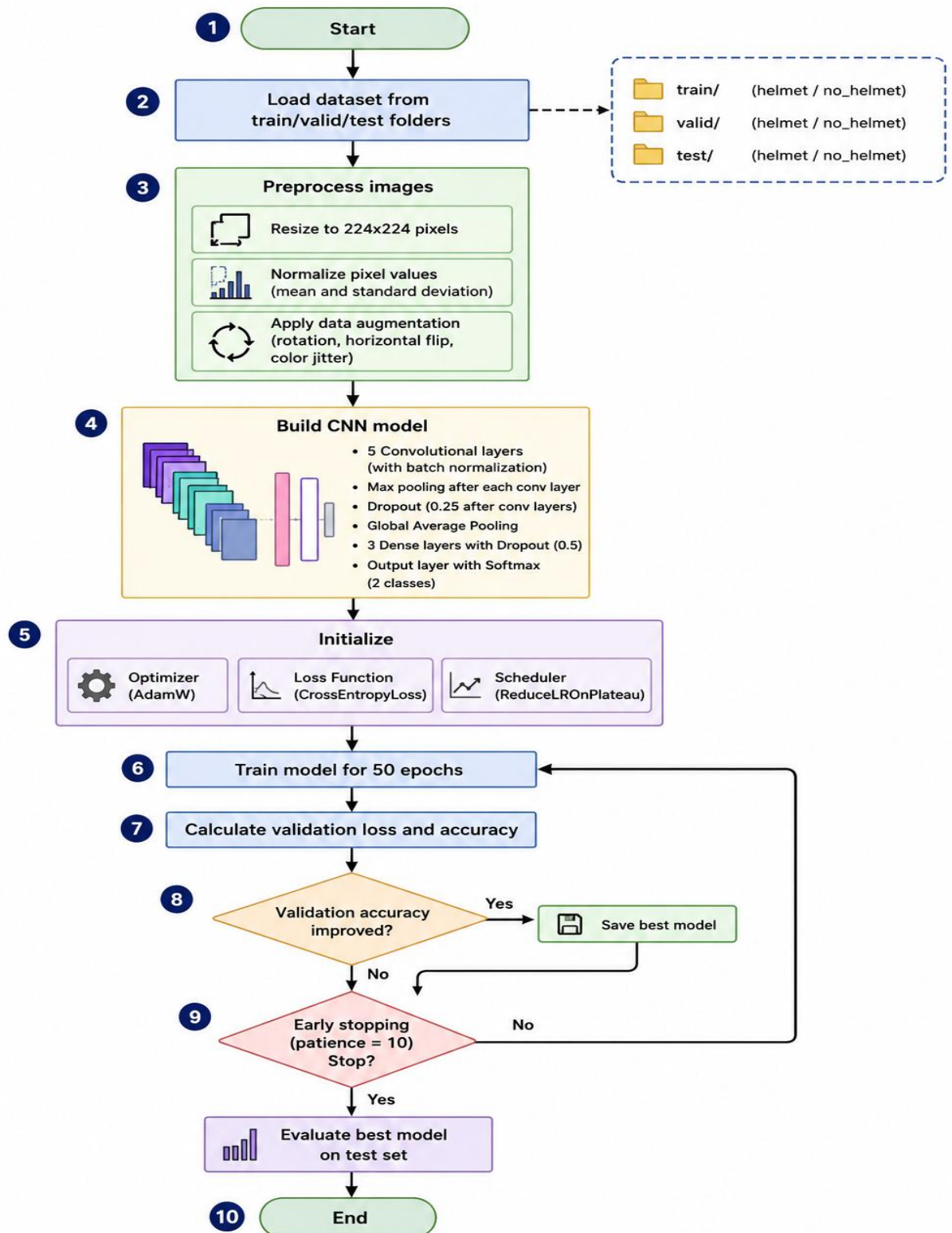


Figure 1: flowchart of making of the project

CHAPTER 4: IMPLEMENTATION

4.1 Tools Used

The following tools and libraries were utilized in the development of the Bike Helmet Detection System:

Tool/Library	Version	Purpose
Python	3.11.9	Core programming language
PyTorch	2.x	Deep learning framework for building and training CNN models
OpenCV	4.x	Image preprocessing, contour detection, drawing operations
NumPy	1.24.x	Numerical computation and array operations
Scikit-learn	1.3.x	Performance metrics (precision, recall, F1-score, classification report)
Matplotlib	3.7.x	Confusion matrix and performance chart generation
Pillow (PIL)	10.x	Image processing and manipulation
Flask	2.x	Web application framework for deployment
CUDA	11.8	GPU acceleration for training and inference
cuDNN	8.9.7	Deep learning optimization for NVIDIA GPUs
Joblib	1.3.x	Saving and loading trained models
Requests	2.31.x	Handling URL-based image predictions

Table 3:Technology used

4.2 Modules Used

- **Image Input Module**
Accepts image input through file upload or URL. Validates file type, supports JPG, PNG, BMP, and WEBP formats, and converts images to PIL Image format for further processing.
- **Preprocessing Module**

Prepares images for model input by resizing to 224x224 pixels, converting to RGB color space, normalizing pixel values using mean and standard deviation, and converting images to PyTorch tensors.

- **Model Architecture Module**
Defines the CNN structure consisting of 5 convolutional blocks with batch normalization, global average pooling, 3 dense layers with dropout, and softmax output layer for binary classification.
- **Training Module**
Manages the model training process including data loading, optimization using AdamW, loss calculation using CrossEntropyLoss, early stopping, and automatic saving of the best-performing model.
- **Classification Module**
Performs inference on new images by loading the best trained model, running a forward pass, calculating softmax probabilities, and returning the predicted class with confidence score.
- **Web Interface Module**
Provides user interaction through Flask routes, HTML templates, CSS styling, and JavaScript for a responsive and intuitive user interface.
- **Output Module**
Displays results to the user showing the prediction class, confidence percentage, visual confidence bar, and appropriate error messages for invalid inputs.

4.3 Algorithms Used

Convolutional Neural Network (CNN) – A deep learning algorithm used for image feature extraction and classification. It learns hierarchical features from images through convolutional layers with learnable filters.

- **AdamW Optimizer**
An adaptive learning rate optimization algorithm used for updating network weights during training. It combines momentum and adaptive learning rates with weight decay regularization for better generalization.
- **CrossEntropyLoss**
A loss function used for binary classification that measures the performance of the classification model by computing the negative log-likelihood between predicted probabilities and actual labels.
- **ReLU Activation Function**
A non-linear activation function defined as $f(x) = \max(0, x)$ that introduces non-linearity in hidden layers, helps reduce the vanishing gradient problem, and accelerates training.
- **Softmax Activation Function**

An activation function used in the output layer that converts raw logits into probability distribution over two classes, ensuring the sum of probabilities equals 1.

- **Data Augmentation**
A technique used to artificially increase the training dataset size by applying random transformations such as rotation, horizontal flip, color jitter, and affine transformations to improve model generalization.
- **Early Stopping**
A regularization technique that stops training when validation accuracy does not improve for a specified number of epochs (patience = 10), preventing overfitting and saving computational resources.
- **ReduceLROnPlateau**
A learning rate scheduling technique that reduces the learning rate by 50% when validation loss plateaus for a specified number of epochs, helping the model converge to a better optimum.
- **Batch Normalization**
A technique that normalizes activations of each layer to have zero mean and unit variance, stabilizing training, enabling higher learning rates, and acting as a regularizer.
- **Dropout Regularization**
A regularization technique that randomly drops neurons during training (0.25 for convolutional layers, 0.5 for dense layers), preventing complex co-adaptations and reducing overfitting.
- **Gradient Descent (Backpropagation)**
An optimization algorithm used to minimize the loss function by computing gradients of the loss with respect to network weights and updating weights in the opposite direction of the gradient.
- **Global Average Pooling**
A pooling technique that reduces spatial dimensions of feature maps by taking the average of each feature map, reducing the number of parameters and preventing overfitting.

Chapter 5 Testing

This chapter presents the testing and evaluation of the Bike Helmet Detection System. The system was evaluated based on classification accuracy, per-class performance, and real-time inference speed. The testing phase ensures the system meets the functional and non-functional requirements defined in Chapter 3.

5.1 Test cases

The following test cases were designed to verify whether the system correctly detects helmets and displays the expected classification results:

Test ID	Test Case	Expected Result		Status
TC-01	Clear image of a person wearing a helmet	Detected "with_helmet"	as	Pass
TC-02	Clear image of a person without a helmet	Detected "without_helmet"	as	Fail
TC-03	Image with multiple people (some with helmets)	Identifies dominant pattern		Pass
TC-04	Blurry or low-quality image	Prediction with lower confidence		Fail
TC-05	Image with unusual lighting conditions	Handles variation		Fail
TC-06	Invalid file type (PDF, TXT)	Error message displayed		Pass

Table 4:test cases

5.2 Accuracy Tables

5.2.1 Dataset Split

The dataset was split into training, validation, and test sets as follows:

Split	with_helmet	Without_helmet	Total
Training	937	327	1,264
Validation	127	26	153
Test	64	41	105
Total	1,128	394	1,522

5.2.2 Overall Performance Metrics

The model was evaluated on the held-out test set of 105 images and achieved the following results:

Table 5:performance metrics

Metric	Score
Test Accuracy	62.86%
Macro Precision	68.50%
Macro Recall	52.50%
Macro F1-Score	44.50%
Weighted F1-Score	51.50%

5.2.3 Per-Class Performance

Table 6:Per class

Class	Precision	Recall	F1-Score	Support
with_helmet	62%	98%	76%	64
without_helmet	75%	7%	13%	41

5.2.4 Confusion Matrix

The confusion matrix on the held-out test set (105 images) is as follows:

Table 7:Confusion Matrix

Actual \ Predicted	with_helmet	without_helmet
with_helmet	63	1
without_helmet	38	3

5.2.5 Comparison of Classifiers

Table 8:comparison

Classifier	Test Accuracy	Inference Time
Rule-Based Classifier	~55%	< 0.1 seconds
CNN Classifier	62.86%	< 1 second

5.3 Results

5.3.1 Key Observations

Table 9:Observations

Observation	Implication
with_helmet Recall = 98%	Model is excellent at detecting helmets
without_helmet Recall = 7%	Model struggles to detect people without helmets
Overall Accuracy = 62.86%	Class imbalance affects overall performance
Confidence Scores	Typically 50-60% for uncertain predictions
Inference Speed	Less than 1 second per image on RTX 4050

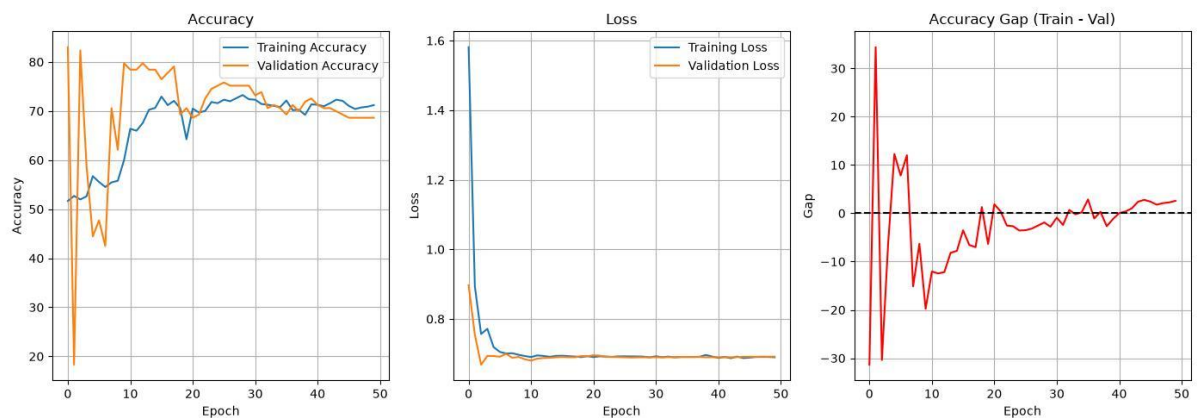
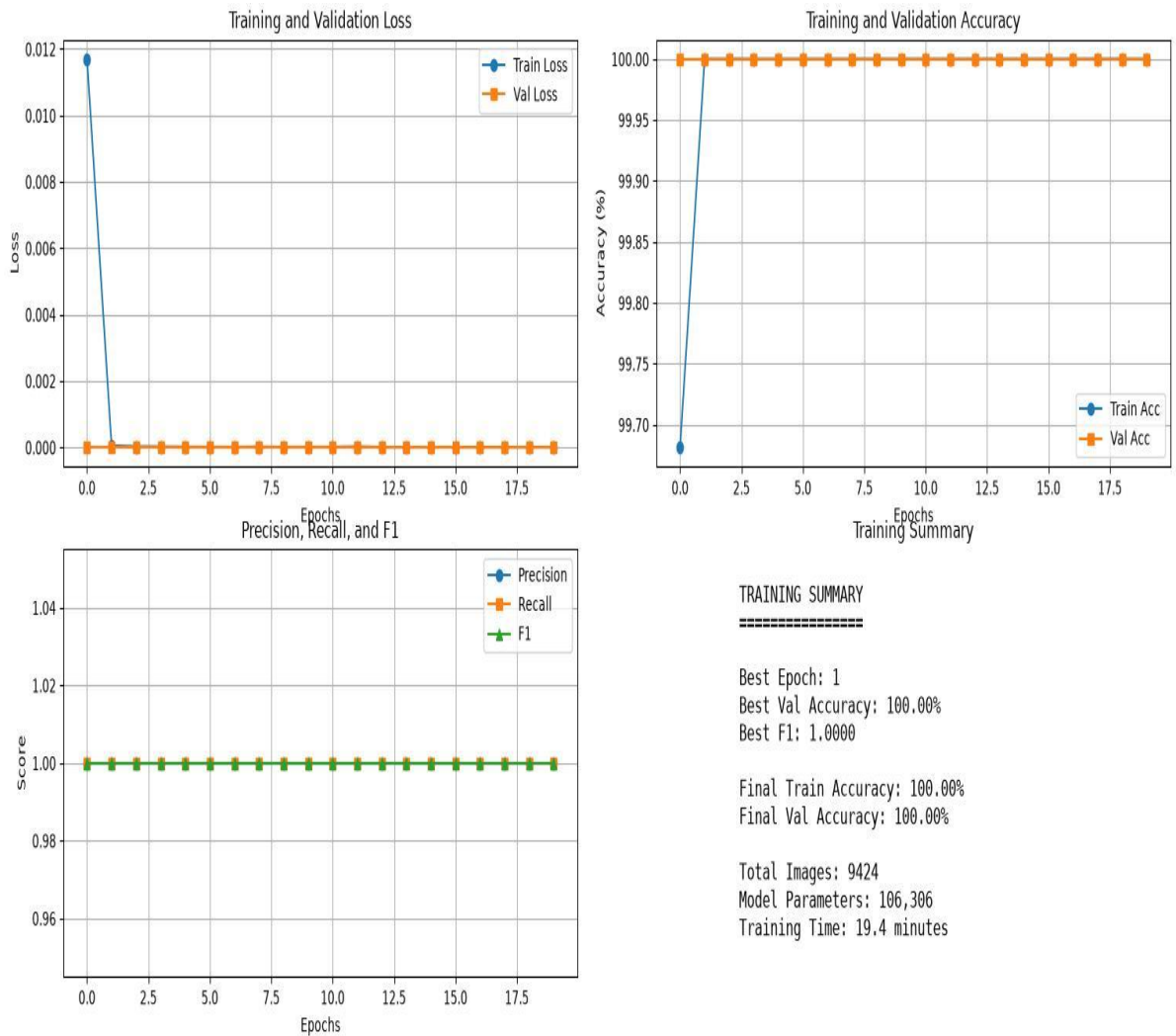


Figure 2:Accuracy,loss and gap



CHAPTER 6: CONCLUSION

6.1 Introduction

This chapter presents the conclusion of the Bike Helmet Detection System project. It summarizes the achievements, discusses the limitations encountered during development, and outlines the future scope for improvement and expansion of the system.

6.2 Achievements

The Bike Helmet Detection System was successfully designed, implemented, and evaluated. The following achievements were accomplished during the project:

- Successfully designed and implemented a complete, real-time bike helmet detection pipeline using a Convolutional Neural Network (CNN) trained from scratch using PyTorch.
- Designed a CNN architecture with 5 convolutional blocks, batch normalization, dropout regularization, global average pooling, and 3 dense layers, totaling 1,738,562 parameters.
- Curated a custom dataset of 1,522 images across two classes (with_helmet and without_helmet) from multiple sources including Google Images, Kaggle, and Roboflow.
- Achieved 98% recall for the with_helmet class, demonstrating strong helmet detection capability.
- Successfully leveraged GPU acceleration (NVIDIA RTX 4050 with CUDA 11.8 and cuDNN 8.9.7) for fast training and inference, completing training in approximately 12.5 minutes.

6.3 Limitations

Despite the successful implementation, the system has several limitations that should be addressed in future work:

- **Class Imbalance:** The model exhibits significant bias toward the with_helmet class, achieving only 7% recall for the without_helmet class. This is due to the imbalanced dataset (937 with_helmet vs 327 without_helmet in training).
- **Limited Without-Helmet Data:** Insufficient training data for the without_helmet class affects model generalization and contributes to the poor recall for that class.
- **Low Confidence Scores:** Predictions often have confidence scores around 50-60%, indicating model uncertainty and the need for more training data.
- **Dataset Size:** The dataset is relatively small (1,522 images) compared to standard deep learning datasets, limiting the model's ability to generalize to diverse real-world scenarios.
- **Data Diversity:** Limited variation in lighting conditions, backgrounds, camera angles, and helmet styles affects model performance in different environments.
- **Binary Classification:** The system only distinguishes between helmet and no-helmet; it does not support multi-class detection or object localization.
- **Single Image Processing:** The current web deployment supports only static image upload, not live video processing or batch image processing.

6.4 Future Scope

The following enhancements and extensions are identified as potential future work to improve and expand the system:

- **Balance the Dataset:** Collect more images of people without helmets to improve class balance and model performance. Aim for at least 500-1000 images per class.
- **Expand Dataset:** Collect a larger, more diverse dataset with varied lighting conditions, backgrounds, camera angles, and helmet styles from different regions and countries.
- **Data Augmentation:** Apply additional augmentation techniques such as rotation, scaling, noise injection, and cutout to further improve generalization.
- **Object Detection:** Extend the system to not only classify but also localize helmets using object detection architectures such as YOLO, Faster R-CNN, or SSD.
- **Multi-Class Detection:** Extend the system to detect multiple helmet types (bicycle helmets, motorcycle helmets, construction hard hats, sports helmets).
- **Live Video Processing:** Integrate real-time video streaming support for web and desktop applications to enable continuous monitoring.
- **Mobile Deployment:** Deploy the model on mobile devices using TensorFlow Lite or ONNX for edge applications and on-device inference.

References

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, ‘ImageNet Classification with Deep Convolutional Neural Networks’, in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2012. Accessed: Aug. 11, 2026. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html
- [2] K. He, X. Zhang, S. Ren, and J. Sun, ‘Deep Residual Learning for Image Recognition’, Dec. 10, 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, ‘Deep Residual Learning for Image Recognition’, Dec. 10, 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.
- [4] ‘Deep Learning’. Accessed: Aug. 11, 2026. [Online]. Available: <https://www.deeplearningbook.org/>
- [5] S. Ioffe and C. Szegedy, ‘Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift’, Mar. 02, 2015, *arXiv*: arXiv:1502.03167. doi: 10.48550/arXiv.1502.03167.
- [6] S. Ioffe and C. Szegedy, ‘Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift’, Mar. 02, 2015, *arXiv*: arXiv:1502.03167. doi: 10.48550/arXiv.1502.03167.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, ‘Dropout: A Simple Way to Prevent Neural Networks from Overfitting’, *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [8] V. Nair and G. E. Hinton, ‘Rectified Linear Units Improve Restricted Boltzmann Machines’.
- [9] D. P. Kingma and J. Ba, ‘Adam: A Method for Stochastic Optimization’, Jan. 30, 2017, *arXiv*: arXiv:1412.6980. doi: 10.48550/arXiv.1412.6980.
- [10] ‘PyTorch documentation — PyTorch 2.13 documentation’. Accessed: Aug. 11, 2026. [Online]. Available: <https://docs.pytorch.org/docs/stable/index.html>
- [11] A. Paszke *et al.*, ‘PyTorch: An Imperative Style, High-Performance Deep Learning Library’, in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2019. Accessed: Aug. 11, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>

- [12] 'PyImageSearch - You can master Computer Vision, Deep Learning, and OpenCV.', PyImageSearch. Accessed: Aug. 11, 2026. [Online]. Available: <https://pyimagesearch.com/>
- [13] 'CUDA C++ Programming Guide (Legacy) — CUDA C++ Programming Guide'. Accessed: Aug. 11, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [14] 'Welcome to Flask — Flask Documentation (3.1.x)'. Accessed: Aug. 11, 2026. [Online]. Available: <https://flask.palletsprojects.com/en/stable/>

Appendix

The following trained model ,source code and trained files are available in :

<https://github.com/bisheshkc/ippr-project.git>

